

Feature: Boiling eggs

as an user

I want to have breakfast

I want machine to boil eggs

Background:

Given machine is standing by

Scenario: run hard egg

Given machine has 10 eggs

And egg setting is hard

And amount of eggs to boil is set to 1

When machine boils eggs

Then there should be 1 boiled egg

And eggs should be hard

It does not look like much, but we are just getting started. Notice how the specification reads almost like a story. Instead of talking about integers, objects and method calls, the specification talks about machines, eggs and boiling. It is much easier for non-coders to read, understand and comment on this kind of text.

At this point, we can already try running our tests, by moving into the *breakfast* directory and issuing the command *behave*. Behave will try to run our first specification, and will fail because we have neither written steps or implemented the actual machine. Because Behave is really helpful, it will tell us how to proceed:

You can implement step definitions for undefined steps with these snippets:

```
@given('machine is standing by')
def impl(context):
    assert False
```

Now we need to define what is to be done when the breakfast machine is standing by and has 10 eggs ready to be boiled. Create the file *eggs.py* in the *steps* directory and add the following code in it:

```
from breakfast import BreakfastMachine
from behave import *
```

```
@given('machine is standing by')
def impl(context):
    context.machine = BreakfastMachine()
```

```
@given('machine has {egg_amount} eggs')
def impl(context, egg_amount):
    context.machine.eggs = int(egg_amount)
```

Steps are the bridge between the specification and the system being tested. They map the natural-language sentences into function calls that the computer can understand.

The first function defines what will happen when there should be a breakfast machine standing by. Let's create a new instance of *BreakfastMachine* and store it in *context*, which is a special object that Behave keeps track of. It is passed from step to step, and can be used to relay information between them. Eventually, we will use it to assert that the specification has been executed correctly.

This defines code that is executed when there is a step '*machine has x eggs*', where x can be anything (in our example it is 10). *{egg_amount}* is automatically parsed and passed as a parameter to the function, which has to have an identically named parameter. Note that the parameters are Unicode strings, and thus need to be converted to integers in our example.

If we were to run Behave at this point, we would get an error message that *BreakfastMachine* cannot be imported. This, of course, is because we have not yet written it. It might feel strange to start coding from this end of the problem (specifications and tests), instead of diving headlong into coding *BreakfastMachine*. The advantage of approaching the task from this direction is that we can first think about how we would like our new object to behave and interact with other objects, and write tests or specifications that capture this. Only after we know how we would like to use the new object do we start writing it.

In order to continue, let's create *BreakfastMachine* in the file *breakfast.py* and save it in the *breakfast* directory. This is the class our client asked us to write and which we want to test:

```
class BreakfastMachine(object):
    def __init__(self):
        super(BreakfastMachine, self).__init__()
        self.eggs = 0
        self.egg_hardness = 'soft'
        self.eggs_to_boil = 0

        self.boiled_eggs = 0
        self.boiled_egg_hardness = None

    def boil_eggs(self):
        pass
```

Steps to implement a hardness setting for eggs and the amount of eggs to boil are quite similar to setting the total amount of eggs available. The difference is that we are not creating a new *BreakfastMachine*, but using the one that has been stored in context earlier. This way, we configure the machine step by step, according to the specification. You can keep running Behave after each addition to *eggs.py* to see what kind of reports it will output. This is a good way of working, because Behave guides you regarding what needs to be done next, in order to fulfil the specification. In *eggs.py*, add the following: